

Mathematics for Linear Regression

Linear Regression

↳ falls under supervised learning.

↳ historic data is labelled and we need to determine the value of dependent variable based on the independent variable.

⇒ SIMPLE LINEAR REGRESSION → Only one predictor variable.
predict the value of dependent variable.

$$y = mx + c.$$

⇒ MULTIPLE LINEAR REGRESSION → More than one predictor variables are used to predict the value of dependent variable.

$$y = m_1x_1 + m_2x_2 + m_3x_3 + \dots + c.$$

for $y = a + bx$.

$$\left[\begin{aligned} a &= \frac{\sum y \sum x^2 - \sum x \sum xy}{n (\sum x^2) - (\sum x)^2} & b &= \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2} \end{aligned} \right]$$

⇒ Uses a modified version of least squares regression to arrive at these values, after a certain number of iterations.

⇒ Uses a mean squared error funcⁿ which is computed using all input-output pairs; calculate the diff b/w the output and input for every pair and square the diff.

After summation, the aggregated value is divide by the number of input-output pairs.

For best fit accuracy, it needs to minimise the mean squared error function, called the 'Cost function'

What "closeness" means?

$$\hat{y}_i = a n_i + b$$

$$\text{error}_i = \hat{y}_i - y_i = \underline{a n_i + b - y_i}$$

→ we don't just look at raw error
we penalize them.

hence they are squared

$$\text{i.e. } (y_i - \hat{y}_i)^2$$

Cost function is basically combining all errors

$$J(a, b) = \frac{1}{n} \sum_{i=1}^n (a n_i + b - y_i)^2$$

small $J(a, b) \rightarrow$ good line

large $J(a, b) \rightarrow$ bad line

i.e. the entire problem is to Minimise $J(a, b)$

We discussed about a, b . To find them, system uses gradient descent.

Now each pair (a, b) defines a line.
each line has an associated error value $J(a, b)$.
so $J(a, b)$ forms a 3D paraboloid surface.

and we are finding global minimum of the surface

Now let's analyze

$$J(a, b) = \frac{1}{n} \sum (ax_i + b - y_i)^2$$

$$(ax_i + b - y_i)^2 = a^2 x_i^2 + b^2 + y_i^2 + 2abx_i - 2ax_i y_i - 2by_i$$

$$J(a, b) = \frac{1}{n} \sum (a^2 x_i^2 + b^2 + y_i^2 + 2abx_i - 2ax_i y_i - 2by_i)$$

$\Rightarrow a, b$ are quadratic.

hence convex $\rightarrow$ single global minimum.

For gradient

$$\frac{\partial J}{\partial a} = \frac{2}{n} \sum (ax_i + b - y_i)x_i \quad \left. \vphantom{\frac{\partial J}{\partial a}} \right\} \text{Gradient wrt 'a'}$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum (ax_i + b - y_i)$$

Now in vector form

Suppose we have $X = [x_1, x_2, x_3, \dots, x_n]^T$.

$$y = [y_1, y_2, \dots, y_n]^T$$

$$\theta = \begin{bmatrix} a \\ b \end{bmatrix}$$

We design a new matrix X' .

$$X' = \begin{bmatrix} n_1 \\ n_2 \\ \vdots \\ n_n \end{bmatrix} \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

i.e. $n \times 2$ matrix.

Input values
of n_i

this generates
intercept b .

Parameter vector

$$\theta = \begin{bmatrix} a \\ b \end{bmatrix} \Rightarrow \begin{matrix} a = \text{slope} \\ b = \text{intercept} \end{matrix}$$

$$\therefore \hat{y} = X' \theta = \begin{bmatrix} n_1 \\ n_2 \\ \vdots \\ n_n \end{bmatrix} \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ \vdots \end{bmatrix} = \begin{bmatrix} a n_1 + b \\ a n_2 + b \\ \vdots \\ a n_n + b \end{bmatrix}$$

$$\text{Error vector} = \hat{y} - y$$

$$e = \begin{bmatrix} a n_1 + b - y_1 \\ a n_2 + b - y_2 \\ \vdots \\ a n_n + b - y_n \end{bmatrix}$$

Error (residual vector)

$$\text{error vector} = \hat{y} - y$$

Square errors

$$e' e \Rightarrow \sum (a n_i + b - y_i)^2$$

Step 2

$$J(\theta) = \frac{1}{n} (X'\theta - y)' (X'\theta - y)$$

Final compact cost funcⁿ

$$J(\theta) = \frac{1}{n} \|X'\theta - y\|^2$$

Gradient $\Rightarrow$ $J(\theta)$

$\hookrightarrow$ To minimise it we compute derivative.

$$\nabla J(\theta) = \frac{2}{n} (X')^T (X'\theta - y)$$

Gradient descent update rule

$$\boxed{\theta = \theta - \alpha \nabla J(\theta)}$$

$\rightarrow$ Subtraction here is to move downhill.

$$\theta = \theta - \frac{2\alpha}{n} (X')^T (X'\theta - y)$$

α here is the learning rate.

Why Gradient Descent works

$\hookrightarrow$ Gradients point in direction of steepest ascent.

$\hookrightarrow$ negative gradient $\rightarrow$ steepest descent.

$\therefore$ each update reduces the error.

$$\text{i.e. } J(\theta_{\text{new}}) < J(\theta_{\text{old}})$$

Final workflow

- 1) Start with data
- 2) Build Matrix (X)
- 3) Assume parameters (θ)
- 4) Predict : $\hat{y} = X' \theta$
- 5) Compute error : $X' \theta - y$
- 6) Compute cost : squared error
- 7) Compute gradient
- 8) Update parameters
- 9) Repeat until stable
- 10) Output best (θ, b)